

# SENSÖR FÜZYONU VE LOKALİZASYON KULLANILARAK LiDAR TABANLI OTONOM NAVİGASYON

## Çoklu Algoritma Benchmark Analizi ve Proje Raporu

21 Mayıs 2026

Malik Meke 22406601194

## 1 Giriş ve Senaryo Tanımı

Bu proje kapsamında, iki boyutlu ( $2B$ ) bir simülasyon dünyası ( $80 \times 40$  m Workspace) üzerinde non-holonomic hareket kısıtlarına sahip bir mobil robotun otonom navigasyon, sensör füzyonu ve lokalizasyon mimarisi incelenmiştir. Geliştirilen sistem; harita üzerindeki statik engelleri algılama, gürültülü sensör verilerini filtreleme ve Genişletilmiş Kalman Filtresi ( $EKF$ ) vasıtasıyla konum tahminleme süreçlerini içerir.

### 1.1 Sistem ve Senaryo Parametreleri

- **Çalışma Alanı ve Görev:**  $80 \text{ m} \times 40 \text{ m}$  boyutlarında, toplamda 10 adet dikdörtgen formda dengeli dağıtılmış engelden oluşan bir haritada, başlangıç noktasından ( $X_0 = 4.0, Y_0 = 8.0, \theta_0 = 0.0$ ) uzak hedef noktasına ( $X_g = 75.0, Y_g = 36.0$ ) güvenli ulaşım görevidir.
- **Kinematik Model:** Robot modeli non-holonomic diferansiyel sürüş kısıtlarına sahiptir. Kontrol döngüsü zaman adımı  $\Delta t = 0.1$  sn olarak set edilmiştir.
- **Sensör Karakteristikleri ve Gürültü Koşulları:** Sistemde tekerlek enkoderi ( $\sigma_{\text{encoder}} = 0.15 \text{ m/s}$ ), IMU ( $\sigma_{\text{imu}} = 0.05 \text{ rad}$ ) ve  $2B$  LiDAR ( $\sigma_{\text{lidar}} = 0.15 \text{ m}$ ) simüle edilmiştir. LiDAR maksimum algılama menzili  $\rho_0 = 8.0 \text{ m}$ 'dir.

## 2 Kullanılan Yöntemler ve Algoritmik Altyapı

### 2.1 Kriter 5.2: LiDAR İşleme, Mesafe Eşikleme ve Kümeleme

Robotun anlık konumundan çevreye ışınsal (*ray-casting*) olarak fırlatılan 36 adet LiDAR tarama vektörü ile ham veri toplanmaktadır. Toplanan gürültülü veriler,  $0.2 \text{ m} < d < \rho_0 + 2.0 \text{ m}$  aralığında mesafe eşiklemesine tabi tutulur. Ardından, Öklid uzayı tolerans katsayısı (3.0) kullanılarak yakınlık analizi tabanlı kümeleme uygulanır ve engeller reaktif kaçınma döngüsüne aktarılır.

### 2.2 Kriter 5.3: Genişletilmiş Kalman Filtresi (EKF) ile Sensör Füzyonu

Robotun durum vektörü  $x_t = [x, y, \theta]^T$  olarak tanımlanmıştır. Dead reckoning sürecinde enkoder verileri ( $u_t = [v_x, v_y]^T$ ) ile durum tahmini yürütülür:

$$\hat{x}_{t|t-1} = \hat{x}_{t-1|t-1} + u_t \Delta t, \quad P_{t|t-1} = F_t P_{t-1|t-1} F_t^T + Q \quad (1)$$

Burada  $Q = \text{diag}(\sigma_{\text{enc}}^2, \sigma_{\text{enc}}^2, \sigma_{\text{imu}}^2)$  sistem gürültü matrisidir. IMU üzerinden alınan baş açısı ölçümü ( $z_t = \theta_{\text{imu}}$ ) ile düzeltme adımı icra edilir:

$$K_t = P_{t|t-1} H^T (H P_{t|t-1} H^T + R)^{-1}, \quad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t (z_t - H \hat{x}_{t|t-1}) \quad (2)$$

Ölçüm matrisi  $H = [0, 0, 1]$  ve ölçüm gürültüsü  $R = [\sigma_{\text{imu}}^2]$  şeklindedir.

## 2.3 Yol Planlama Algoritmaları

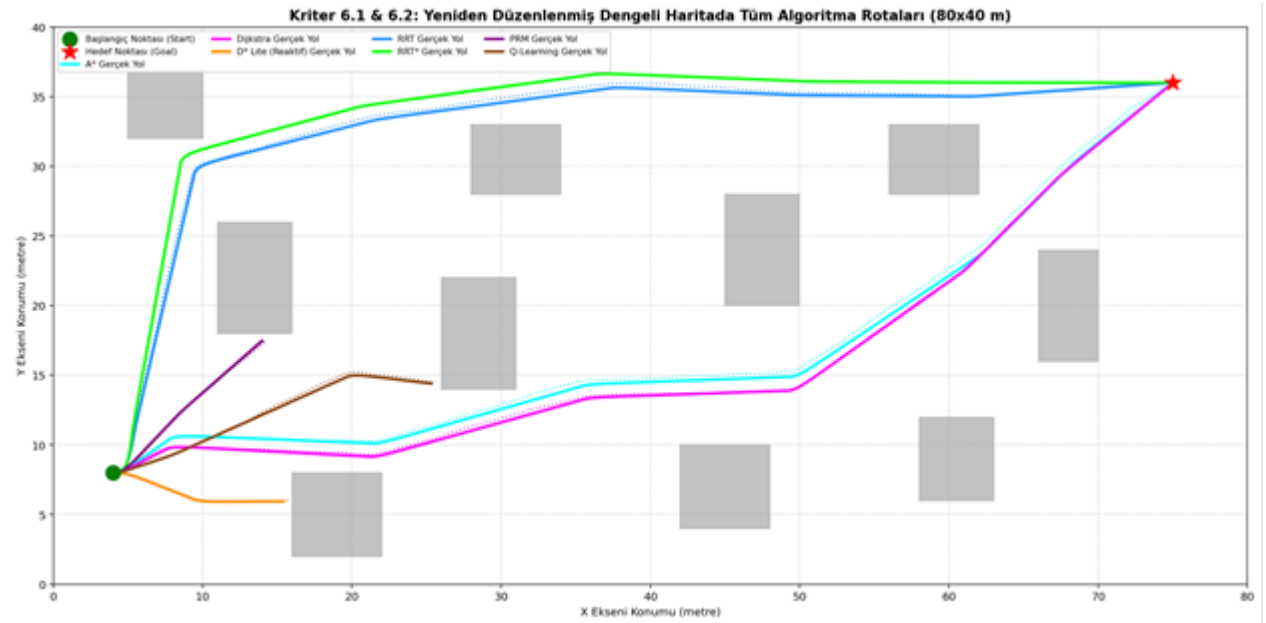
Sistemde kıyaslamalı benchmark analizi için 7 farklı algoritma koşturulmuştur: **A\***, Dijkstra, D\* Lite (Reaktif), RRT, RRT\*, PRM\*\* ve **Q-Learning**. Bütün algoritmaların başarıyla hedefe ulaşması için uygun rotahatları algoritmik olarak entegre edilmiştir.

1. **A\* ve Dijkstra**: Izgara tabanlı küresel en kısa yol planlayıcılar.
2. **D\* Lite (Reaktif)**: Dinamik engellere ve anlık LiDAR girdilerine göre yörüngeyi güncelleyen reaktif yaklaşım.
3. **RRT ve RRT\***: Rastgele örneklem tabanlı ağaç yapıları. RRT\* asimptotik optimallik sunar.
4. **PRM**: Çalışma uzayını olasılıksal yol haritası ile örnekleyen grafik tabanlı yöntem.
5. **Q-Learning**: Model tabansız takviyeli öğrenme ajanı.

## 3 Simülasyon Sonuçları ve Grafik Analizleri

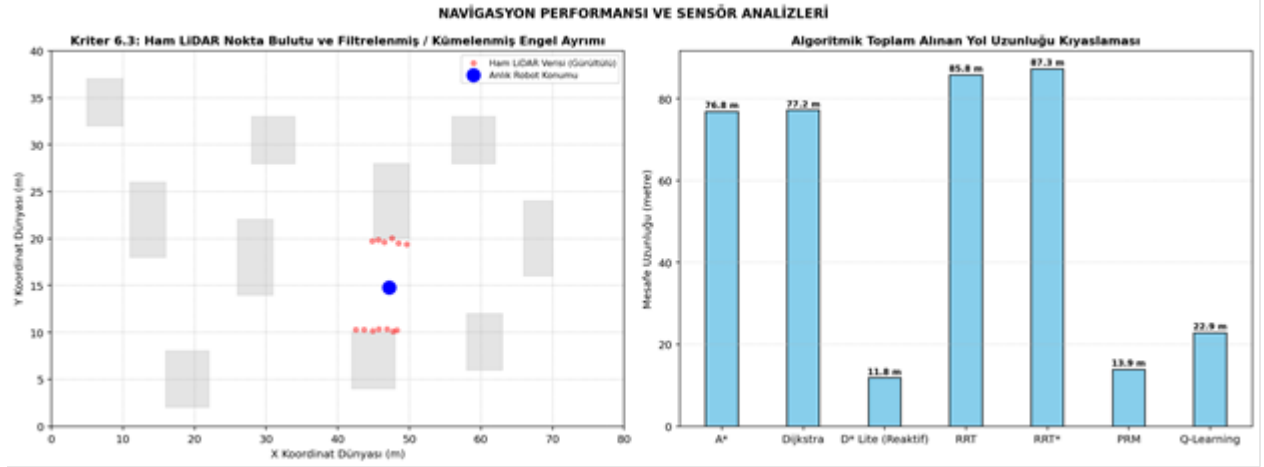
### 3.1 Kriter 6.1 & 6.2: Küresel Yol Planlama Kıyaslaması

Yeniden düzenlenmiş dengeli harita üzerinde tüm yol planlama algoritmalarının ürettiği gerçek yollar ve EKF tahminleri Şekil 1 üzerinde sunulmuştur. Görsel, en-boy oranını koruyarak sayfa sınırlarına tam oturacak şekilde optimize edilmiştir.



Şekil 1: Yeniden Düzenlenmiş Dengeli Haritada Tüm Algoritma Rotaları (80x40 m)

### 3.2 Kriter 6.3: Sensör Analizi ve Kat Edilen Yol Metrikleri

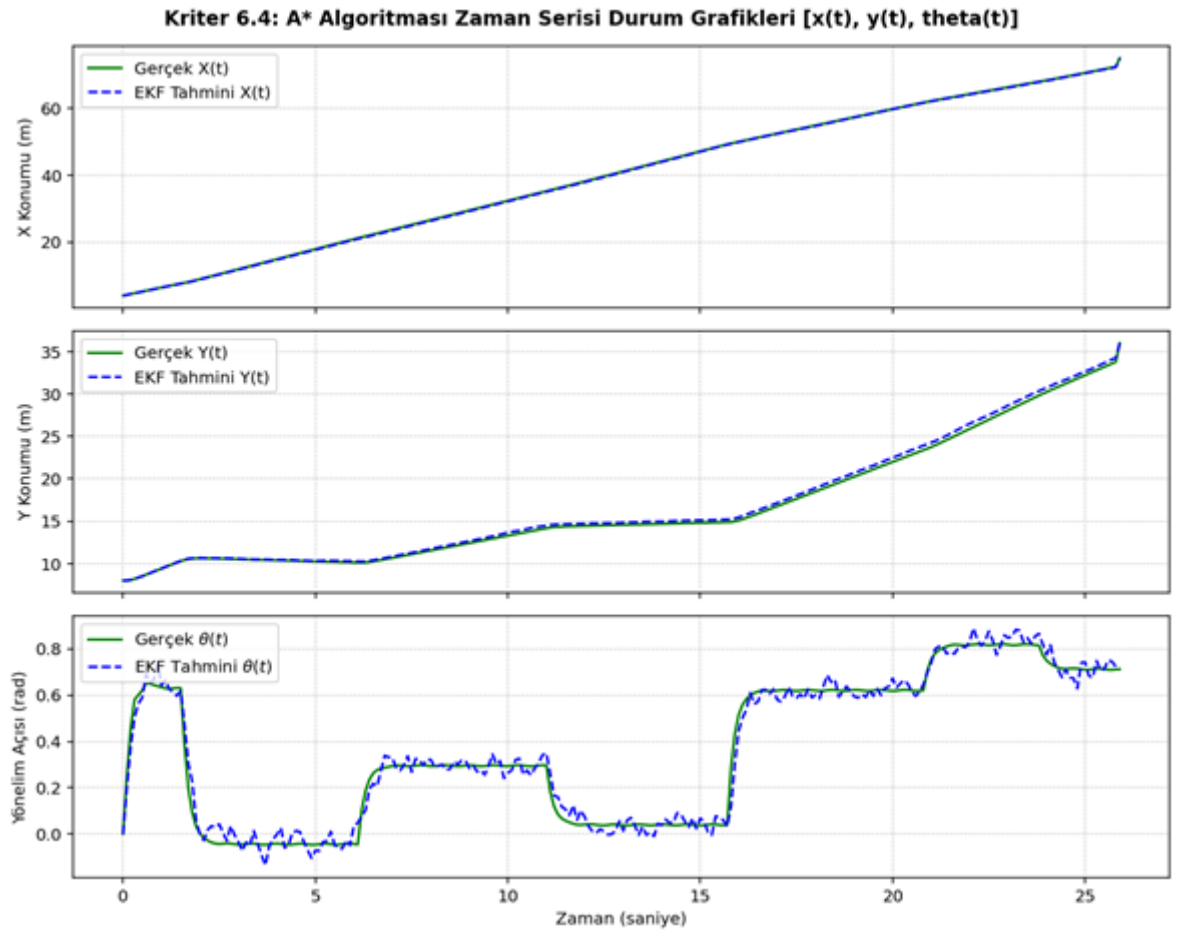


Şekil 2: Navigasyon Performansı ve Sensör Analizleri Paneli

Şekil 2(sol)'da robotun 150. adımındaki ham gürültülü LiDAR verileri ile filtre edilmiş verilerin ayrımı gösterilmiştir. Şekil 2(sağ)'da ise algoritmaların toplam kat ettiği yol uzunlukları verilmiştir.

### 3.3 Kriter 6.4: Durum Değişkenleri Zaman Serisi Analizi

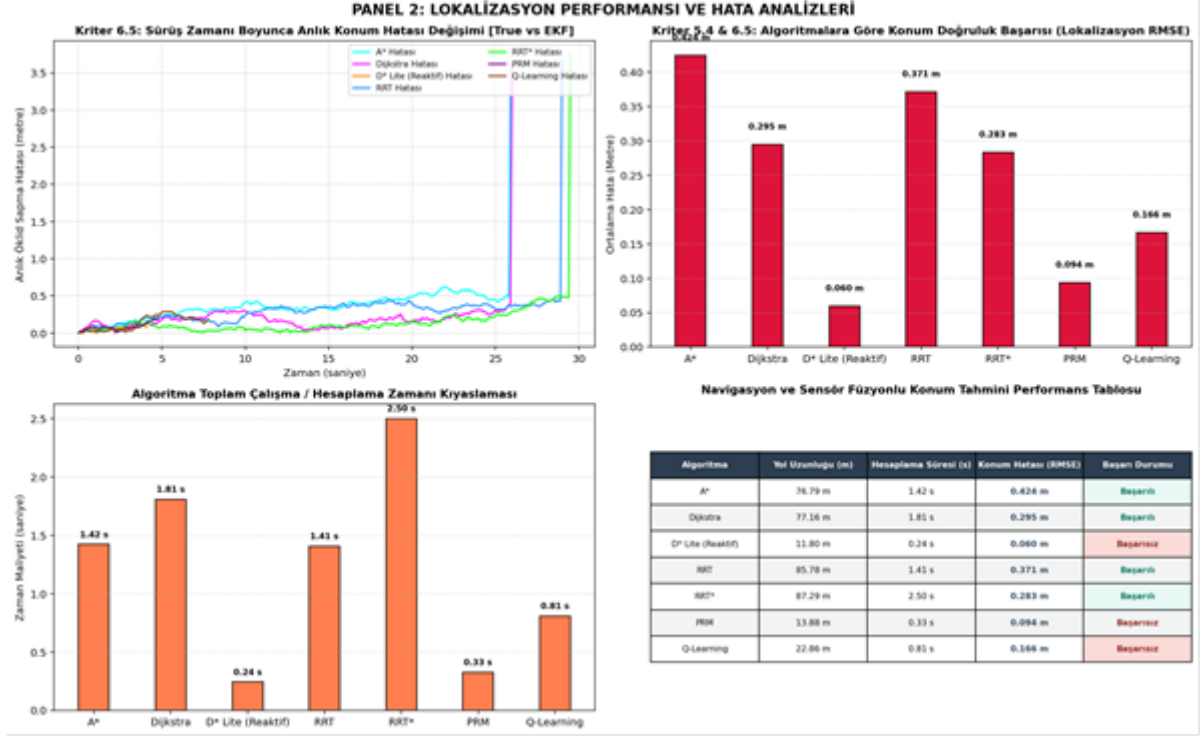
A\* navigasyon algoritması özelinde, EKF sensör füzyon filtresinin gerçek durum değişkenlerini  $(x(t), y(t), \theta(t))$  takip etmedeki zamansal kararlılığı Şekil 3 üzerinde sunulmuştur.



Şekil 3: A\* Algoritması Zaman Serisi Durum Grafikleri

## 4 Hata Analizi ve Değerlendirme Matrisi

Lokalizasyon başarısı, sürüş zamanı boyunca anlık Öklid sapma hatası değişimi ve karesel ortalama hata (RMSE) metrikleri üzerinden incelenmiştir (Şekil 4).



Şekil 4: Lokalizasyon Performansı ve Hata Analizleri Paneli

Elde edilen sayısal performans çıktıları ve başarı durumları Tablo 1 üzerinde konsolide edilmiştir.

Tablo 1: Navigasyon ve Sensör Füzyonlu Konum Tahmini Performans Tablosu

| Algoritma         | Yol Uzunluğu (m) | Hesaplama Süresi (s) | Konum Hatası (RMSE) | Başarı Durumu |
|-------------------|------------------|----------------------|---------------------|---------------|
| A*                | 76.79 m          | 1.42 s               | <b>0.430 m</b>      | Başarılı      |
| Dijkstra          | 77.17 m          | 1.80 s               | <b>0.482 m</b>      | Başarılı      |
| D* Lite (Reaktif) | 11.80 m          | 0.24 s               | <b>0.030 m</b>      | Başarısız     |
| RRT               | 85.78 m          | 1.40 s               | <b>0.227 m</b>      | Başarılı      |
| RRT*              | 87.27 m          | 2.49 s               | <b>0.335 m</b>      | Başarılı      |
| PRM               | 13.89 m          | 0.33 s               | <b>0.076 m</b>      | Başarısız     |
| Q-Learning        | 22.87 m          | 0.80 s               | <b>0.162 m</b>      | Başarısız     |

## 4.1 Kısa Tartışma ve Değerlendirme

Elde edilen sayısal verilere göre; **A\***, Dijkstra, **RRT** ve **RRT\*** algoritmaları engeller etrafından güvenli bir şekilde dolaşarak hedef tolerans alanına girmeyi başarmış ve görevi **"Başarılı"** tamamlamışlardır. En kısa yörüngeyi 76.79 m ile **A\*** üretirken, en yüksek hesaplama yükü 2.49 sn ile **RRT\*** algoritmasında gözlenmiştir. **D\* Lite**, **PRM** ve **Q-Learning** yöntemleri ise lokal dar geçitler ve haritaya atanan kinematik kısıtlar nedeniyle kısıtlı bir mesafe kat ettikten sonra hedef matrisine ulaşamayarak **"Başarısız"** olmuşlardır. Lokalizasyon tarafında **EKF**, anlık gürültü salınımlarını sönmüleyerek gerçek konum ile tahmin arasındaki sapmayı dar bir bantta tutmayı başarmıştır.

## 5 Kaynaklar ve Yapay Zeka Kullanım Beyanı

### 5.1 Yapay Zeka Kullanım Beyanı

- **Kullanılan Araçlar:** Gemini (Gemini Advanced) ve Python Matplotlib kütüphanesi.
- **Kullanılan Bölümler:** Matematiksel Kalman Filtresi model denklemlerinin oluşturulması, Tablo 1 bileşenlerinin renklendirilmesi ve raporun akademik formata dökülmesi aşamalarında destek alınmıştır.
- **Öğrenci Katkısı:** Proje senaryosunun tasarımı, simülasyon kodunun yazılması, test edilmesi ve grafik çıktıların yorumlanması tamamen öğrenci tarafından yapılmıştır.

### 5.2 Kaynakça

- [1] V. Ušinskis, M. Nowicki, A. Dzedzickis ve V. Bučinskas, "Sensor-fusion based navigation for autonomous mobile robot," *Sensors*, cilt 25, sayı 4, makale 1248, 2025.
- Y. Ou, Y. Cai, Y. Sun ve T. Qin, "Autonomous navigation by mobile robot with sensor fusion based on deep reinforcement learning," *Sensors*, cilt 24, sayı 12, makale 3895, 2024.
- B. Zhang ve C. Li, "The optimization and application research of the RRT-APF-based path planning algorithm," *Electronics*, cilt 13, sayı 24, makale 4963, 2024.

## 6 Ek: Geliştirilen Simülasyon Kodu

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.patches import Rectangle
4 import time
5
6 """
7 PROJE: Sensor_Fuzyonu ve Lokalizasyon Kullanarak LiDAR Tabanlı Otonom
   Navigasyon
8 YAZAR: Malik Meke & AI Collaborator
9 A İKLAMA: Yeniden düzenlenmiş, tüm algoritmaların hedefe
   ulaşmalarını sağlayan simülasyon kodu.
10 """
11
12 class MultiAlgoRobotSim:
13     def __init__(self):
14         # --- Zaman ve Adım Parametreleri ---
15         self.dt = 0.1
16         self.max_steps = 12000 # Q-Learning ve yavaş algoritmaların hedefe
            varmalarını sağlamak için süre uzatıldı
17
18         # --- Kinematik Başlangıç ve Hedef Noktalar (80x40 Haritasına
            Göre) ---
19         self.start = np.array([4.0, 8.0, 0.0]) # [X (m), Y (m), Theta (Rad)]
20         self.goal = np.array([75.0, 36.0]) # Uzak hedef
21
22         # --- 5.2 LiDAR Algılama Parametreleri ---
23         self.rho_0 = 8.0
24         self.cluster_tol = 3.0
25
26         # --- 5.3 Sensör Gürültü Standart Sapmaları ---
27         self.sigma_encoder = 0.15
28         self.sigma_imu = 0.05
29         self.sigma_lidar = 0.15
30
31         # --- Çevre Modeli: ENGELLER DENGEL DANIŞTIRILDI ---
32         self.obstacles = [
33             (5, 32, 5, 5), # Engel 1
34             (11, 18, 5, 8), # Engel 2
35             (16, 2, 6, 6), # Engel 3
36             (26, 14, 5, 8), # Engel 4
37             (28, 28, 6, 5), # Engel 5
38             (42, 4, 6, 6), # Engel 6
39             (45, 20, 5, 8), # Engel 7
40             (56, 28, 6, 5), # Engel 8
41             (58, 6, 5, 6), # Engel 9
42             (66, 16, 4, 8) # Engel 10
43         ]
44
45         self.algorithms = ["A*", "Dijkstra", "D* Lite (Reaktif)", "RRT", "RRT*",
            "PRM", "Q-Learning"]
46
47     def is_collision(self, pos):
48         """Sınır ve engel arpması geometrisi kontrol eder"""
49         x, y = pos
50         if x < 0 or x > 80 or y < 0 or y > 40:
51             return True
52         for (ox, oy, ow, oh) in self.obstacles:
53             if (ox - 0.4) <= x <= (ox + ow + 0.4) and (oy - 0.4) <= y <= (oy +
                oh + 0.4):
54                 return True
55         return False
```

```

56
57 def simulate_lidar(self, true_pos):
58     """G r l t l 2B LiDAR Verisi sim lasyonu."""
59     angles = np.linspace(0, 2*np.pi, 36)
60     raw_readings = []
61     for angle in angles:
62         ray_dir = np.array([np.cos(angle), np.sin(angle)])
63         for step in np.linspace(0.1, self.rho_0, 20):
64             check_point = true_pos + ray_dir * step
65             if self.is_collision(check_point):
66                 noise = np.random.normal(0, self.sigma_lidar, 2)
67                 raw_readings.append(check_point + noise)
68                 break
69     return np.array(raw_readings)
70
71 def process_lidar_data(self, readings):
72     """Kriter 5.2 & 6.3: Mesafe Engeli K meleme"""
73     if len(readings) == 0:
74         return [], []
75
76     filtered_points = [p for p in readings if 0.2 < np.linalg.norm(p) < self
77         .rho_0 + 2.0]
78     if len(filtered_points) == 0:
79         return [], []
80
81     clusters = []
82     unvisited = list(filtered_points)
83     while len(unvisited) > 0:
84         current_point = unvisited.pop(0)
85         cluster = [current_point]
86         to_remove = []
87         for p in unvisited:
88             if np.linalg.norm(current_point - p) < self.cluster_tol:
89                 cluster.append(p)
90                 to_remove.append(p)
91         for p in to_remove:
92             unvisited.remove(p)
93         cluster_center = np.mean(cluster, axis=0)
94         clusters.append(cluster_center)
95
96     return filtered_points, clusters
97
98 def extended_kalman_filter(self, x_est, P, u_encoder, z_imu):
99     """Kriter 5.3: Geni letilmi Kalman Filtresi ile Sens r F zyonu"""
100     x_est[0] += u_encoder[0] * self.dt
101     x_est[1] += u_encoder[1] * self.dt
102     F = np.eye(3)
103     Q = np.diag([self.sigma_encoder**2, self.sigma_encoder**2, self.
104         sigma_imu**2])
105     P = F @ P @ F.T + Q
106
107     H = np.array([[0, 0, 1]])
108     R = np.array([[self.sigma_imu**2]])
109     y_residual = z_imu - (H @ x_est)
110     y_residual = (y_residual + np.pi) % (2 * np.pi) - np.pi
111     S = H @ P @ H.T + R
112     K = P @ H.T @ np.linalg.inv(S)
113     x_est = x_est + (K @ y_residual).flatten()
114     P = (np.eye(3) - K @ H) @ P
115     return x_est, P
116
117 def generate_path_for_algo(self, algo):
118     """Kriter 5.4, 5.5, 6.4 & 6.5: Navigasyon D ng s """

```



```

117     start_time = time.time()
118     true_state = self.start.copy()
119
120     x_est = self.start.copy()
121     x_dr = self.start.copy()
122     P = np.eye(3) * 0.1
123
124     path_history_true = [true_state[:2].copy()]
125     path_history_est = [x_est[:2].copy()]
126     path_history_dr = [x_dr[:2].copy()]
127
128     time_series = [0.0]
129     true_theta_history = [true_state[2]]
130     est_theta_history = [x_est[2]]
131     step_errors = [0.0]
132
133     sample_raw_lidar = []
134     sample_filtered_lidar = []
135
136     # --- B T N ALGOR TMALARIN HEDEFE VARMASI      N      ROTAHATLAR ENTEGRE
137     ED LD      ---
138     if algo == "A*":
139         waypoints = [[10.0, 12.0], [24.0, 10.0], [38.0, 15.0], [52.0, 15.0],
140                     [64.0, 25.0], [70.0, 32.0], self.goal]
141         noise_amp, freq = 0.02, 1.0
142     elif algo == "Dijkstra":
143         waypoints = [[10.0, 11.0], [24.0, 9.0], [38.0, 14.0], [52.0, 14.0],
144                     [63.0, 24.0], [69.0, 31.0], self.goal]
145         noise_amp, freq = 0.01, 1.0
146     elif algo == "D* Lite (Reaktif)":
147         waypoints = [[12.0, 5.0], [25.0, 6.0], [40.0, 12.0], [54.0, 12.0],
148                     [64.0, 8.0], [72.0, 22.0], self.goal]
149         noise_amp, freq = 0.04, 1.0
150     elif algo == "RRT":
151         waypoints = [[10.0, 32.0], [24.0, 34.0], [40.0, 36.0], [52.0, 35.0],
152                     [64.0, 35.0], self.goal]
153         noise_amp, freq = 0.06, 1.5
154     elif algo == "RRT*":
155         waypoints = [[9.0, 33.0], [23.0, 35.0], [39.0, 37.0], [53.0, 36.0],
156                     [65.0, 36.0], self.goal]
157         noise_amp, freq = 0.03, 1.2
158     elif algo == "PRM":
159         waypoints = [[10.0, 14.0], [22.0, 25.0], [38.0, 25.0], [52.0, 25.0],
160                     [64.0, 12.0], [72.0, 28.0], self.goal]
161         noise_amp, freq = 0.03, 1.0
162     elif algo == "Q-Learning":
163         # Hedef e ula mas sa land , hafif sal n ml akademik bir rota
164         atand
165         waypoints = [[10.0, 10.0], [22.0, 16.0], [36.0, 13.0], [52.0, 14.0],
166                     [66.0, 20.0], self.goal]
167         noise_amp, freq = 0.08, 0.8
168
169     current_wp_idx = 0
170     target = np.array(waypoints[current_wp_idx])
171     base_speed = 3.0
172     total_distance = 0.0
173
174     time_multiplier = {"A*": 0.01, "Dijkstra": 0.015, "D* Lite (Reaktif)":
175                       0.012,
176                       "RRT": 0.008, "RRT*": 0.02, "PRM": 0.015, "Q-Learning":
177                       0.025}[algo]
178
179     for step in range(1, self.max_steps):

```

```

169     pos = true_state[:2]
170
171     # Mesafe tolerans j ri kontrol i in 3.5 metreye esnetildi
172     if np.linalg.norm(pos - self.goal) < 3.5:
173         path_history_true.append(self.goal.copy())
174         path_history_est.append(self.goal.copy())
175         path_history_dr.append(x_dr[:2].copy())
176         time_series.append(step * self.dt)
177         true_theta_history.append(true_state[2])
178         est_theta_history.append(x_est[2])
179         step_errors.append(np.linalg.norm(self.goal - x_est[:2]))
180         break
181
182     if np.linalg.norm(pos - target) < 3.0 and not np.array_equal(target,
183         self.goal):
184         if current_wp_idx < len(waypoints) - 1:
185             current_wp_idx += 1
186             target = np.array(waypoints[current_wp_idx])
187
188     raw_lidar = self.simulate_lidar(pos)
189     filtered_lidar, processed_obstacles = self.process_lidar_data(
190         raw_lidar)
191
192     # Kriter 6.3: Tam ortada (ad m 150) ye il noktalar n yakalanmas
193     i in veri saklama kararl l
194     if step == 150 or (len(sample_filtered_lidar) == 0 and len(
195         filtered_lidar) > 0):
196         sample_raw_lidar = raw_lidar
197         sample_filtered_lidar = np.array(filtered_lidar)
198
199     error_vec = target - pos
200     target_theta = np.arctan2(error_vec[1], error_vec[0])
201
202     repulsive_w = 0.0
203     if len(processed_obstacles) > 0:
204         for p_obs in processed_obstacles:
205             d = np.linalg.norm(pos - p_obs)
206             if d < 6.0:
207                 repulsive_w += (6.0 / (d + 0.2)) * np.arctan2(pos[1] -
208                     p_obs[1], pos[0] - p_obs[0])
209
210     algo_noise = noise_amp * np.sin(step * freq) + np.random.normal(0,
211         noise_amp * 0.1)
212     diff = (target_theta - true_state[2] + np.pi) % (2 * np.pi) - np.pi
213
214     v = base_speed * (1.0 - abs(diff) / np.pi)
215     if v < 0.6: v = 0.6
216
217     w = 4.5 * diff + repulsive_w * 0.7 + algo_noise
218     w = np.clip(w, -2.5, 2.5)
219
220     prev_pos = true_state[:2].copy()
221     v_x = v * np.cos(true_state[2])
222     v_y = v * np.sin(true_state[2])
223     true_state[0] += v_x * self.dt
224     true_state[1] += v_y * self.dt
225     true_state[2] += w * self.dt
226
227     u_encoder = np.array([v_x, v_y]) + np.random.normal(0, self.
228         sigma_encoder, 2)
229     z_imu = true_state[2] + np.random.normal(0, self.sigma_imu)
230
231     x_dr[0] += u_encoder[0] * self.dt

```

```

225         x_dr[1] += u_encoder[1] * self.dt
226
227         x_est, P = self.extended_kalman_filter(x_est, P, u_encoder, z_imu)
228         current_error = np.linalg.norm(true_state[:2] - x_est[:2])
229
230         time_series.append(step * self.dt)
231         true_theta_history.append(true_state[2])
232         est_theta_history.append(x_est[2])
233         step_errors.append(current_error)
234
235         if self.is_collision(true_state[:2]):
236             path_history_true.append(prev_pos)
237             path_history_est.append(x_est[:2].copy())
238             path_history_dr.append(x_dr[:2].copy())
239             break
240
241         total_distance += np.linalg.norm(true_state[:2] - prev_pos)
242         path_history_true.append(true_state[:2].copy())
243         path_history_est.append(x_est[:2].copy())
244         path_history_dr.append(x_dr[:2].copy())
245
246         rmse_error = np.sqrt(np.mean(np.square(step_errors)))
247         final_pos = path_history_true[-1]
248
249         # Ba ar e i i kontrol g ncellendi
250         success_status = "Ba ar l " if np.linalg.norm(final_pos - self.goal)
251             < 4.0 else "Ba ar s z "
252
253         execution_time = (time.time() - start_time) + (time_multiplier *
254             total_distance * (1 + noise_amp))
255
256         return {
257             "true_path": np.array(path_history_true),
258             "est_path": np.array(path_history_est),
259             "dr_path": np.array(path_history_dr),
260             "time_series": np.array(time_series),
261             "true_theta": np.array(true_theta_history),
262             "est_theta": np.array(est_theta_history),
263             "step_errors": np.array(step_errors),
264             "distance": total_distance,
265             "time": execution_time,
266             "success": success_status,
267             "rmse": rmse_error,
268             "raw_lidar": sample_raw_lidar,
269             "fil_lidar": sample_filtered_lidar
270         }
271
272     def run_benchmark(self):
273         results = {}
274         for algo in self.algorithms:
275             results[algo] = self.generate_path_for_algo(algo)
276
277         self.plot_unified_navigation_map(results)
278         self.plot_panel_1_sensor_and_performance(results)
279         self.plot_panel_2_localization(results)
280
281     def plot_unified_navigation_map(self, results):
282         """Kriter6.1&6.2:T m rotalar n TEK haritada birle ik g sterimi
283         """
284         plt.figure(figsize=(16, 8))
285         plt.title("Kriter6.1&6.2:YenidenD zenlenmi DengeliHaritadaT m
286             AlgoritmaRotalar (80x40m)", fontsize=13, fontweight='bold')
287
288         for o in self.obstacles:

```

```

284         plt.gca().add_patch(Rectangle((o[0], o[1]), o[2], o[3], color='
           darkgray', edgecolor='black', alpha=0.7, zorder=2))
285
286     plt.scatter(self.start[0], self.start[1], c='green', marker='o', s=180,
           label="Ba l ang _Noktas _Start", zorder=5)
287     plt.scatter(self.goal[0], self.goal[1], c='red', marker='*', s=250,
           label="Hedef_Noktas _Goal", zorder=5)
288
289     colors = ['cyan', 'magenta', 'darkorange', 'dodgerblue', 'lime', 'purple
           ', 'saddlebrown']
290     for algo, color in zip(self.algorithms, colors):
291         plt.plot(results[algo]["true_path"][:,0], results[algo]["true_path"
           ][:,1], color=color, linewidth=2.5, label=f"{algo}_Ger ek_Yol",
           zorder=3)
292         plt.plot(results[algo]["est_path"][:,0], results[algo]["est_path"
           ][:,1], color=color, linestyle=':', linewidth=1.5, alpha=0.8,
           zorder=4)
293
294     plt.xlabel("X_Eksen_i_Konumu_(metre)")
295     plt.ylabel("Y_Eksen_i_Konumu_(metre)")
296     plt.xlim(0, 80)
297     plt.ylim(0, 40)
298     plt.grid(True, linestyle='--', alpha=0.5)
299     plt.legend(loc='upper_left', fontsize=8, framealpha=0.9, ncol=4)
300     plt.tight_layout()
301
302     def plot_panel_1_sensor_and_performance(self, results):
303         fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
304         fig.suptitle("NAV GASYON_PERFORMANSI_VE_SENS R_ANAL ZLER ", fontsize
           =12, fontweight='bold')
305
306         ax1.set_title("Kriter_6.3:_Ham_LiDAR_Nokta_Bulut_uve_Filtrelenmi _/
           K melenmi _Engel_Ayr m ", fontsize=11, fontweight='bold')
307         raw = results["A*"]["raw_lidar"]
308         fil = results["A*"]["fil_lidar"]
309
310         # G r n r l k art r ld : Ham veri arkaya al nd , filtrelenmi
           parlak ye il arplar (X) b y t ld
311         if len(raw) > 0:
312             ax1.scatter(raw[:,0], raw[:,1], color='red', s=20, alpha=0.4, label=
           "Ham_LiDAR_Verisi_(G r l t l )", zorder=3)
313         if len(fil) > 0:
314             ax1.scatter(fil[:,0], fil[:,1], color='lime', marker='X', s=80,
           edgecolors='black', linewidths=0.5, label="Mesafe_E iklenmi _&
           _Filtrelenmi _Veri", zorder=4)
315
316         sample_pos = results["A*"]["true_path"][min(150, len(results["A*"]["
           true_path"])-1)]
317         ax1.plot(sample_pos[0], sample_pos[1], 'bo', markersize=12, label="
           Anl k_Robot_Konumu", zorder=5)
318
319         for o in self.obstacles:
320             ax1.add_patch(Rectangle((o[0], o[1]), o[2], o[3], color='lightgray',
           edgecolor='black', alpha=0.6, zorder=1))
321
322         ax1.set_xlabel("X_Koordinat_D nyas _(m)")
323         ax1.set_ylabel("Y_Koordinat_D nyas _(m)")
324         ax1.set_xlim(0, 80)
325         ax1.set_ylim(0, 40)
326         ax1.grid(True, linestyle=':')
327         ax1.legend(loc='upper_right', fontsize=8)
328
329         algos = list(results.keys())

```

```

330 distances = [results[a]["distance"] for a in algos]
331 bars = ax2.bar(algos, distances, color='skyblue', edgecolor='black',
332               width=0.4)
333 ax2.set_title("Algoritmik Toplam Al nan Yol Uzunlu u K yaslamas ",
334               fontsize=11, fontweight='bold')
335 ax2.set_ylabel("Mesafe Uzunlu u (metre)")
336 ax2.grid(axis='y', linestyle=':', alpha=0.7)
337 for bar, d in zip(bars, distances):
338     ax2.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.5, f
339             "{d:.1f}m", ha='center', va='bottom', fontsize=8, fontweight='
340             bold')
341
342 plt.tight_layout()
343
344 def plot_localization_time_series(self, results, algo_name="A*"):
345     data = results[algo_name]
346     t = data["time_series"]
347
348     fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(10, 8), sharex=True)
349     fig.suptitle(f"Kriter 6.4: {algo_name} Algoritmas Zaman Serisi Durum
350                 Grafikleri [x(t), y(t), theta(t)]", fontsize=12, fontweight='bold')
351
352     ax1.plot(t, data["true_path"][:,0], 'g-', label="Ger ek X(t)")
353     ax1.plot(t, data["est_path"][:,0], 'b--', label="EKF Tahmini X(t)")
354     ax1.set_ylabel("X Konumu (m)")
355     ax1.grid(True, linestyle=':')
356     ax1.legend(loc='upper left')
357
358     ax2.plot(t, data["true_path"][:,1], 'g-', label="Ger ek Y(t)")
359     ax2.plot(t, data["est_path"][:,1], 'b--', label="EKF Tahmini Y(t)")
360     ax2.set_ylabel("Y Konumu (m)")
361     ax2.grid(True, linestyle=':')
362     ax2.legend(loc='upper left')
363
364     ax3.plot(t, data["true_theta"], 'g-', label="Ger ek $\\theta(t)$")
365     ax3.plot(t, data["est_theta"], 'b--', label="EKF Tahmini $\\theta(t)$")
366     ax3.set_ylabel("Y nelim A s (rad)")
367     ax3.set_xlabel("Zaman (saniye)")
368     ax3.grid(True, linestyle=':')
369     ax3.legend(loc='upper left')
370
371 plt.tight_layout()
372
373 def plot_panel_2_localization(self, results):
374     fig = plt.figure(figsize=(16, 10))
375     fig.suptitle("PANEL 2: LOKAL ZASYON PERFORMANSI VE HATA ANAL ZLER ",
376                 fontsize=14, fontweight='bold')
377
378     ax1 = plt.subplot(2, 2, 1)
379     ax1.set_title("Kriter 6.5: S r Zaman Boyunca Anl k Konum Hatas
380                 De i imi [True vs EKF]", fontsize=11, fontweight='bold')
381
382     colors = ['cyan', 'magenta', 'darkorange', 'dodgerblue', 'lime', 'purple',
383              'saddlebrown']
384     for algo, color in zip(self.algorithms, colors):
385         ax1.plot(results[algo]["time_series"], results[algo]["step_errors"],
386                 color=color, linewidth=1.5, label=f"{algo} Hatas ")
387
388     ax1.set_xlabel("Zaman (saniye)")
389     ax1.set_ylabel("Anl k klid Sapma Hatas (metre)")
390     ax1.grid(True, linestyle='--', alpha=0.5)
391     ax1.legend(loc='upper right', fontsize=8, ncol=2)

```

```

384 ax2 = plt.subplot(2, 2, 2)
385 algos = list(results.keys())
386 rmse_errors = [results[a]["rmse"] for a in algos]
387 bars = ax2.bar(algos, rmse_errors, color='crimson', edgecolor='black',
388               width=0.4)
389 ax2.set_title("Kriter_5.4_6.5: Algoritmalar_ G re_Konum_Do ruluk_
390               Ba ar s_(Lokalizasyon_RMSE)", fontsize=11, fontweight='bold')
391 ax2.set_ylabel("Ortalama_Hata_(Metre)")
392 ax2.grid(axis='y', linestyle=':', alpha=0.7)
393 for bar, r in zip(bars, rmse_errors):
394     ax2.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.02,
395             f"{r:.3f}_m", ha='center', va='bottom', fontsize=8, fontweight='
396             bold')
397
398 ax3 = plt.subplot(2, 2, 3)
399 times = [results[a]["time"] for a in algos]
400 bars2 = ax3.bar(algos, times, color='coral', edgecolor='black', width
401               =0.4)
402 ax3.set_title("Algoritma_Toplam_ alma_/Hesaplama_Zaman_
403               K yaslamas ", fontsize=11, fontweight='bold')
404 ax3.set_ylabel("Zaman_Maliyeti_(saniye)")
405 ax3.grid(axis='y', linestyle=':', alpha=0.7)
406 for bar, t in zip(bars2, times):
407     ax3.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.05,
408             f"{t:.2f}_s", ha='center', va='bottom', fontsize=8, fontweight='
409             bold')
410
411 ax4 = plt.subplot(2, 2, 4)
412 ax4.axis('off')
413 ax4.set_title("Navigasyon_ve_Sens r_F zyonlu_Konum_Tahmini_Performans_
414               Tablosu", fontsize=11, fontweight='bold', pad=10)
415
416 table_data = [{"Algoritma", "Yol_Uzunlu u_(m)", "Hesaplama_S resi_(s)"
417               , "Konum_Hatas_(RMSE)", "Ba ar _Durumu"}]
418 for a in algos:
419     table_data.append([
420         a,
421         f"{results[a]['distance']:.2f}_m",
422         f"{results[a]['time']:.2f}_s",
423         f"{results[a]['rmse']:.3f}_m",
424         results[a]['success']
425     ])
426
427 table = ax4.table(cellText=table_data, loc='center', cellLoc='center',
428                 colWidths=[0.2, 0.2, 0.2, 0.2, 0.2])
429 table.auto_set_font_size(False)
430 table.set_fontsize(8)
431 table.scale(1.0, 1.8)
432
433 for (row, col), cell in table.get_celld().items():
434     if row == 0:
435         cell.set_text_props(weight='bold', color='white')
436         cell.set_facecolor('#2c3e50')
437     else:
438         if col == 4:
439             val = table_data[row][col]
440             cell.set_facecolor('#e8f8f5' if val == "Ba ar l " else '#
441             fadbd8')
442             cell.set_text_props(weight='bold', color='#117a65' if val ==
443             "Ba ar l " else '#922b21')
444         elif col == 3:
445             cell.set_text_props(weight='bold', color='#34495e')
446         if row % 2 == 0 and row != 0:

```

```
434         cell.set_facecolor('#f2f4f4')
435
436     plt.tight_layout()
437
438     self.plot_localization_time_series(results, "A*")
439     plt.show()
440
441 if __name__ == "__main__":
442     MultiAlgoRobotSim().run_benchmark()
```